Auto Ticket Classification Using Flow Diagram

Team ID: SWTID-2026-3818 | College: Sri Paramakalyani College

Team Lead: Ramalakshmi Nachiyar | Team Members: Sandhiya S Bala Krishnammal S Kalyani M

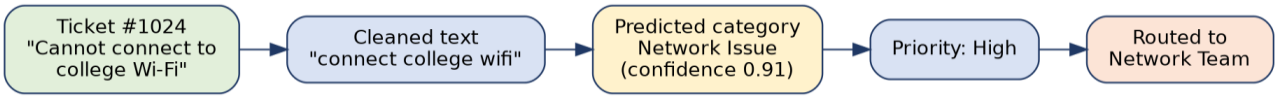
Project Description

Auto Ticket Classification Using Flow Diagram applies machine learning and Natural Language Processing to automatically sort IT support tickets. A user types the problem in a simple web form. The system cleans the text, converts it into TF-IDF features and uses a trained classifier to predict the ticket category (for example Network, Hardware, Software, Account & Access, Email) along with a confidence score and priority.

The ticket is then routed to the responsible support team, and the system generates a flow diagram that shows the journey of the ticket step by step. Tickets with low confidence are sent to an agent for manual review, and the agent's corrections are stored to retrain the model. An admin dashboard shows ticket statistics and model performance.

Scenarios

Scenario 1: A student writes "Unable to connect to college Wi-Fi from the library". The system predicts Network Issue with high confidence, marks the priority High, routes the ticket to the Network Team and shows the flow diagram below.

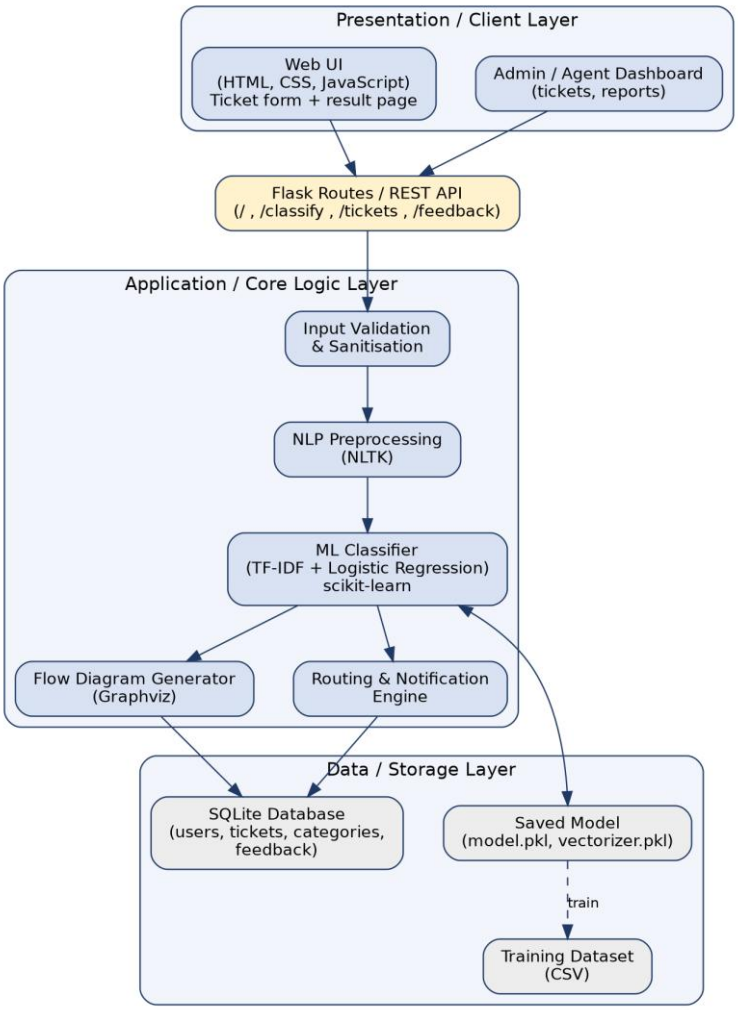


Scenario 2: An employee writes "Forgot my password and account is locked". The system classifies it as Account & Access, priority Medium, and routes it to the Access Management Team.

Scenario 3: A vague ticket says "Something is not working". The model confidence is below the threshold, so the ticket is sent to an agent for manual labelling, and the corrected label is saved for retraining.

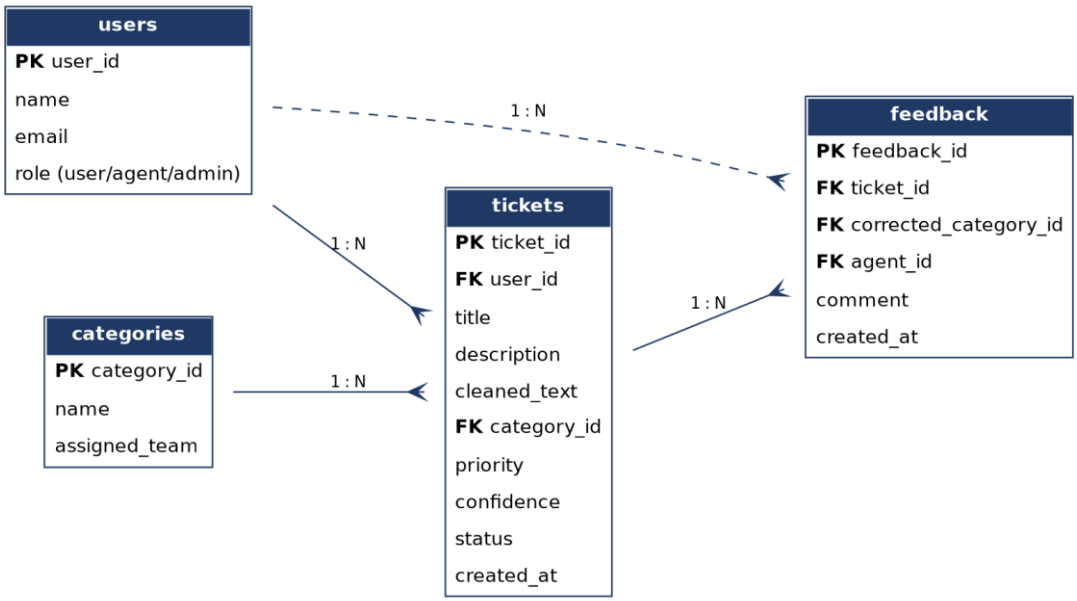
Technical Architecture

The system uses a modular three-layer architecture. The presentation layer (HTML, CSS, JavaScript) collects tickets and shows results. The application layer (Flask) validates input, preprocesses text, runs the ML classifier, routes tickets and generates flow diagrams with Graphviz. The data layer stores tickets and feedback in SQLite and keeps the trained model and datasets as files.



ER Diagram

A database (SQLite) is used, so the entity-relationship diagram is given below. A user can raise many tickets; each ticket belongs to one predicted category; and each ticket can receive feedback from agents.



Pre-requisites

- Python programming proficiency (Python documentation)
- Flask framework knowledge (Flask documentation)

- Machine learning basics: text classification, TF-IDF, scikit-learn
- HTML, CSS and JavaScript skills
- SQL / SQLite basics
- Graphviz installation and basics
- Version control with Git and GitHub
- Development environment setup: Python 3.10+, VS Code, Jupyter Notebook

Project Workflow

- **Milestone 1: Data Collection and Preprocessing:** collect the ticket dataset, explore it and clean the text.
- **Milestone 2: Model Building:** create TF-IDF features, train and compare classifiers, evaluate and save the best model.
- **Milestone 3: Backend Development (app.py):** build Flask routes, validation, prediction, routing and database functions.
- **Milestone 4: Frontend and Flow Diagram:** design the ticket form, result page, dashboard and the Graphviz flow diagram generator.
- **Milestone 5: Testing and Deployment:** API and load testing, then local deployment (optional public link).
- **Milestone 6: Conclusion:** documentation, demo and future scope.

Milestone 1: Data Collection and Preprocessing

Activity 1.1: Collect the dataset

Download or prepare a labelled IT support ticket dataset in CSV format ([dataset name and link]). Each row should contain the ticket text and its category. Keep a small sample in the data/ folder.

Activity 1.2: Explore and clean the data

Check for missing values and duplicates, view the number of tickets per category and fix class imbalance if needed (for example by combining very rare categories).

Activity 1.3: Text preprocessing

Convert text to lowercase, remove punctuation, numbers and stop-words, and apply lemmatisation using NLTK. Split the data into training and testing sets (for example 80:20).

Milestone 2: Model Building

Activity 2.1: Feature extraction

Convert the cleaned text into numerical features using the TF-IDF vectoriser (unigrams and bigrams).

Activity 2.2: Train and compare models

Train Logistic Regression, Multinomial Naive Bayes and Linear SVM. Compare them using accuracy, precision, recall and weighted F1-score.

Activity 2.3: Evaluate and save

Plot the confusion matrix, select the best model, set the confidence threshold and save the model and vectoriser with joblib. Results: [add accuracy and F1-score here].

Milestone 3: Backend Development (app.py)

Create the Flask application with these routes:

- / : home page with the ticket form
- /classify (POST) : validates input, cleans text, predicts category, priority and confidence, routes the ticket, stores it and returns the result
- /tickets : list of tickets for agents and admin

- /feedback (POST) : saves the agent's corrected category
- /dashboard : summary charts

Milestone 4: Frontend and Flow Diagram

Build the HTML templates (index, result, tickets, dashboard) with CSS and JavaScript. Implement build_flow_diagram() using Graphviz to draw the path: Ticket received, Text cleaned, Category predicted, Priority assigned, Routed to team. Save the image in static/ and show it on the result page.

Milestone 5: Testing and Deployment

Test the endpoints with Postman, write unit tests with pytest and run load tests with Locust (see the Performance Testing document). Run the app locally with Flask; optionally expose it with Ngrok or deploy it on a cloud platform.

Milestone 6: Conclusion

The project shows that machine learning can reduce manual effort in ticket handling, and that a flow diagram makes the process transparent to users and managers. Future work includes transformer models, multilingual support and integration with tools such as email or Jira.